



FOR SOFTWARE ENGINEERS: THE TEN MINUTES NOBODY DOCUMENTS

Claude Code: Install

From nothing installed to your first commit

The right command, run in the wrong shell, fails with an error that never says which shell it wanted.

This is the half of getting started that every guide skips, because it seems too obvious to write down. It is also where people lose an afternoon.

Four things are on this page: the one line for your shell, the error that does not say what is wrong, the command that tells you whose job upgrading is, and the five commands that are your entire first week.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- The three native install lines, and how to tell which one is yours
- The two error messages, decoded, and what your prompt already told you
- Which routes update themselves and which make it your job
- Verifying the install before you build anything on top of it
- The five commands that cover the first week

Out of scope

- Prompting technique, which is the rest of the series
- Any claim about a plan or price. Those move faster than this page can
- IDE and editor integrations, which each deserve their own walkthrough
- Enterprise gateway and policy setup

If you read one section, read the wrong-shell one. It is the only thing here that is actively costing somebody time today.

01 The wrong shell, decoded

The token '&&' is not a valid statement separator

You are in PowerShell. You ran the CMD line

'irm' is not recognized as an internal or external command

You are in CMD. You ran the PowerShell line

PS C:\> in front of your prompt

PowerShell. Use the irm line

C:\> with no PS

CMD. Use the curl install.cmd line

macOS, Linux, WSL

`curl -fsSL https://claude.ai/install.sh | bash`

Native Windows, additionally

Install Git for Windows, or it falls back to PowerShell as its shell

Neither message says "wrong shell". Your prompt already did, and that is the whole fix.

03 One line, and it has to be the right one

macOS, Linux and Windows Subsystem for Linux: curl the install script and pipe it to bash.

Windows PowerShell: irm the install script and pipe it to iex.

Windows CMD: curl the installer down, run it, delete it.

The native route is the recommended one, and it has a property the others do not: it updates itself in the background.

The native install is presented as the recommended method, and native installations automatically update in the background to keep you on the latest version.

Claude Code docs, Overview, checked September 2026

Git for Windows is recommended on native Windows so the Bash tool can be used; without it PowerShell is used as the shell tool instead, and WSL setups do not need it.

Claude Code docs, Overview, checked September 2026

04 The package manager routes, and what they cost

Homebrew has a cask, WinGet has a package, and apt, dnf and apk cover Debian, Fedora, Red Hat and Alpine.

None of them auto-update. Upgrading becomes your job, and if you forget you can sit several versions behind for months without knowing.

Homebrew has one more wrinkle worth choosing deliberately. There are two casks: the plain one tracks the stable channel, roughly a week behind, skipping releases that had a major regression. The at-latest one ships the day a release does.

Homebrew and WinGet installations do not auto-update and must be upgraded manually.

Claude Code docs, Overview, checked September 2026

The claude-code cask tracks the stable release channel, typically about a week behind and skipping releases with major regressions, while claude-code@latest receives new versions as soon as they ship.

Claude Code docs, Overview, checked September 2026

05 Prove it worked before you build on it

`claude --version` prints a version number followed by Claude Code in brackets. No brackets, or command not found, and either the install did not finish or your path found something else first.

Then run `claude doctor`, which almost nobody does. It reports the install method it believes it used, and whether auto-updates are switched on.

That answers the question you will actually have in three months, which is: which route did I take, and am I responsible for upgrading this thing.

`claude --version` prints a version number followed by "(Claude Code)".

Claude Code docs, Overview, checked September 2026

`claude doctor` prints read-only installation and settings diagnostics without starting a session, including install health.

Claude Code docs, Overview, checked September 2026

Run on the machine that built this sheet, doctor printed "Config install method: native" and "Auto-updates: enabled", which is the specific output this page relies on.

Run on the machine that built this sheet, not quoted

06 Logging in, and the behaviour that confuses API users

Type `claude` on its own and it prompts you the first time, completing in your browser. Subscription, Console, or a cloud provider.

The one to know about: if `ANTHROPIC_API_KEY` is already set in your shell, you get no login prompt at all. It asks you to approve that key instead. If you expected a browser window and got a question about a key, that variable is why.

Credentials are stored, so it is a one off. Type `/login` inside a session to switch accounts later.

If the `ANTHROPIC_API_KEY` environment variable is set, the login prompt is skipped and you are asked to approve the key instead.

Claude Code docs, Overview, checked September 2026

Once logged in, credentials are stored and you will not need to log in again.

Claude Code docs, Overview, checked September 2026

07 Where the first ten minutes actually go

Not prompting. The folder. Start a session and there is a line above the prompt showing the version, the model, and the working directory.

Read the working directory line every single time. It reads your project files as it needs them, and only the ones where you started it. Start in your home folder, ask what this project does, and the tool looks broken when it was simply pointed at nothing.

Two that nobody discovers alone: Tab completes a command you have started, and the up arrow walks your history, exactly like a shell.

The prompt is shown with the version, current model and working directory above it, and files are read as needed rather than added manually.

Claude Code docs, Overview, checked September 2026

08 Who decides whether that edit is allowed

On Pro, Max and Team, interactive terminal sessions start in auto mode: a classifier reviews the actions instead of you, so most edits and commands happen without stopping to ask. On other plans, sessions start in manual mode.

Your settings, or your organisation, can change which mode a session starts in.

Shift and Tab together cycles the mode of the session you are in. Learn it now, because the moment you want it is the moment something is about to touch a file you care about.

Auto mode is the built-in starting permission mode for interactive terminal sessions on Pro, Max and Team plans; on other plans Manual mode is the starting mode.

Claude Code docs, Permission modes, checked September 2026

Pressing Shift+Tab at any time switches the permission mode of the current session.

Claude Code docs, Permission modes, checked September 2026



09 Ask before you instruct, and the order is the lesson

Start with questions: what does this project do, where is the main entry point, what technologies does it use. You get a useful answer AND a look at how it explores your codebase, which tells you whether you started it in the right place.

Then change something small and obvious, so you can tell instantly whether it worked.

Then use it for the thing people take months to discover, which is git. What files have I changed. Commit my changes with a descriptive message. Create a branch. You are talking to it the way you would talk to a colleague looking at the same screen.

Git operations are conversational, covering changed files, commits, branches, history and merge conflicts.

Claude Code docs, Overview, checked September 2026

Your first week, in ten checks

1. Which shell am I actually in, and does my prompt show PS?
2. Did `claude --version` print the brackets?
3. What did Claude Doctor say my install method was?
4. Are auto-updates its job or mine?
5. On Homebrew, am I on the stable cask or at-latest?
6. On native Windows, is Git for Windows installed?
7. Is `ANTHROPIC_API_KEY` set in my shell, and did I mean it to be?
8. Am I reading the working directory line before I type?
9. Do I know which permission mode this session started in?
10. Can I get out with `/exit` without looking it up?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

Quickstart

<https://code.claude.com/docs/en/quickstart>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.